Lead Intelligence Agent

A multi-agent pipeline for lead qualification, outreach, and governance — from ML score to a reviewed, human-approved message

---

Live Dashboard: coffra-marketing-dashboard.streamlit.app

Project	P7 · Coffra Lead Intelligence Agent
Author	Sebastian Kradyel
Date	July 2026
Repository	github.com/sebikradyel1-svg/coffra-marketing-automation
Stack	Python · Anthropic Claude API · XGBoost + SHAP · LangChain + FAISS · Streamlit
Agents	4 chained: Qualification → Outreach → Governance → Human Approval
Status	v1.0 — agents + RAG grounding + dashboard integration + case study

Executive Summary

P1 built a lead scoring model and a static three-tier segmentation table (Sales-Ready / Warm MQL / Cold) mapping score to action. That table was a specification, not a system — someone still had to read a score, decide a tier, draft outreach, check it for compliance, and get a human to approve it. P7 operationalizes that specification into a live, agentic pipeline: four chained agents that call the P1 model as a tool, draft grounded outreach, enforce a governance rubric, and stop at a human-approval checkpoint before anything would reach a real inbox.

The architecture deliberately separates generation from evaluation: a Qualification Agent scores and routes, an Outreach Agent drafts, and a distinct Governance Reviewer — with no authority to rewrite, only to verdict — checks the draft claim-by-claim against approved brand sources before a human ever sees it. This split mirrors current practice in AI evaluation and governance: the agent that generates content should not be the same agent that grades it.

This case study documents the system honestly, including three real issues found and fixed during development — a model artifact mismatch, a governance rubric that missed a grounding failure on its first pass, and a JSON parsing edge case — and a calibration of the Governance Reviewer against a 50-draft hand-labeled test set (100% recall, 88.2% precision), because a measured claim about how well a component works is worth more than an architectural description of it.

Key Outcomes

Component	Outcome
Pipeline architecture	4 chained agents: Qualification (tool use) → Outreach (RAG) → Governance (claim verification) → Human approval
Qualification model	P1's XGBoost lead scorer wrapped as a Claude tool call; SHAP top-3 factors surfaced in every decision
Tier thresholds	HOT ≥ 0.70 (outreach) · WARM 0.40-0.70 (nurture) · COLD < 0.40 (disqualify)
Grounding	RAG (LangChain + FAISS, local sentence-transformer embeddings) restricts outreach claims to approved brand sources
Governance	Claim-by-claim decomposition, APPROVE/REVISE verdict; any unsupported claim forces REVISE — no holistic pass
Governance calibration	100% recall · 88.2% precision · 92% accuracy against a 50-draft hand-labeled test set (Aug 2026)
Live verification	All 3 test leads (HOT/WARM/COLD) run end-to-end locally and on Streamlit Cloud; both APPROVE and REVISE verdicts observed live
Deployment	Integrated as page 10 of the existing Coffra Streamlit dashboard (P2)

Problem Statement and Approach

The challenge

A trained model that outputs a probability is not a decisioning system. Between 'the model says 0.98' and 'a rep sends a message' sit several distinct judgments: which tier does this score map to, what does that mean for the next action, what should the outreach say, is what it says actually true, and should a human sign off. Collapsing all of that into a single 'do everything' LLM call makes each judgment unauditable — if the output is wrong, there is no way to tell whether scoring, drafting, or reviewing failed.

P7 addresses this by decomposing the pipeline into agents with narrow, single-purpose responsibilities, each independently inspectable and independently testable, connected by structured JSON contracts rather than free-form handoffs.

Scope decisions

Decision	Choice	Rationale
Agent decomposition	4 narrow agents, not 1 mega-agent	Isolates failure points; each agent has an auditable, single responsibility
Grounding strategy	RAG over approved data/brand_sources.md	Prevents outreach from inventing product or brand claims

Decision	Choice	Rationale
Governance rigor	Forced claim-by-claim JSON decomposition, not a holistic pass/fail	An impressionistic reviewer misses mixed true/false sentences (see Rigor and Limitations)
Human checkpoint	Simulated UI approval; no message is ever actually sent	Portfolio safety — demonstrates the governance pattern without live-send risk
Test leads	3 fixed synthetic leads (HOT / WARM / COLD)	Reproducible, deterministic verification of all three tiers and both governance verdicts

Methodology principles

The same disciplines used across P1-P5 apply here: honest documentation of failures rather than only successes, reproducible test leads with fixed feature vectors, versioned prompts and code in Git, and — specific to this project — an explicit separation between the agent that generates content and the agent that evaluates it. That separation is a deliberate design decision, not an implementation detail: it is the same principle behind independent AI evaluation and governance layers now standard practice in production AI marketing systems, where the model producing an output should not also be the sole judge of its correctness.

Agent Architecture

Four agents are chained in sequence, each receiving only the structured output of the previous step — no shared mutable state, no implicit context:

Agent	Role	Technology
1. Qualification	Calls the P1 XGBoost model as a tool (score_lead), interprets the score plus its top-3 SHAP factors, and decides tier (HOT/WARM/COLD) and next action on fixed 0.70/0.40 thresholds	Claude tool use · XGBoost · SHAP TreeExplainer
2. Outreach	Drafts a personalized first-touch message for HOT leads only, grounded in retrieved brand talking points so it cannot invent product claims	Claude · RAG (LangChain + FAISS, all-MiniLM-L6-v2 embeddings)
3. Governance	Reviews the draft against a strict rubric: decomposes it into individual claims, checks each against approved sources, evaluates brand voice / banned patterns / compliance / personalization, returns APPROVE or REVISE	Claude · structured JSON verdict, no rewrite authority

Agent	Role	Technology
4. Human approval	UI checkpoint: REVISE blocks and routes to revision; APPROVE surfaces Approve & send / Send back for revision — no message is ever actually transmitted	Streamlit dashboard (page 10)

Why the Qualification Agent calls a tool instead of guessing

The Qualification Agent's system prompt explicitly instructs it to always call score_lead before deciding a tier — never to estimate the score from intuition — and to fall back to WARM (not disqualify) if the tool call fails. This keeps the tier decision grounded in the actual XGBoost output and its SHAP factors rather than in the language model's own, unverifiable judgment of the lead.

Why the Outreach Agent is RAG-grounded

data/brand_sources.md — the approved talking points, segmentation table, and verified results from P1 — is chunked (300 characters, 40 overlap) and embedded locally with sentence-transformers, indexed in FAISS. Each outreach draft retrieves only the top-3 chunks most relevant to the lead's specific qualification signal, and the system prompt instructs the agent that any claim not traceable to those chunks must not be stated. This is the same grounding principle as the Governance Reviewer, applied one step earlier — reducing the number of ungrounded claims the reviewer has to catch downstream, without removing the need for the review itself.

Rigor and Limitations

Three real issues surfaced during development. They are documented here in full because a process that catches and fixes its own failures is stronger evidence of engineering judgment than a system presented as having none.

Issue 1 — Model artifact mismatch

The initially saved model artifacts did not reproduce the predictions recorded in models/sample_predictions_v1.csv — correlation with true_label was 0.10, effectively random. The mismatch was diagnosed, the model was retrained and re-exported, and the final artifact reached a correlation of 0.834 against the same held-out predictions. A saved .joblib file that loads without error is not proof it is the right model — this is why score_lead's artifacts are checked against a recorded prediction sample rather than trusted on load.

Issue 2 — Governance grounding miss (V1 → V2)

The first version of the Governance Reviewer's system prompt asked it to evaluate the outreach draft holistically. On a test draft mixing a real fact with an ungrounded addition in the same sentence ("Coffra is a D2C specialty coffee brand" — true — chained with "built for personal ritual, not standardized" — an invented positioning claim), the V1 reviewer approved the whole sentence, missing the ungrounded half. The fix was structural, not cosmetic: the required JSON output was restructured to force claim-by-claim

decomposition — every distinct assertion, even ones joined by a comma in the same sentence, must be listed and checked individually before any verdict is issued. Any claim marked unsupported now automatically forces a hard flag and a REVISE verdict; the reviewer can no longer average a bad claim away against good ones.

The fix was confirmed 3/3 consistent in repeated testing, and confirmed again independently during live verification on Streamlit Cloud, where the Governance Reviewer returned a real REVISE verdict — catching an unsupported "sales-ready" claim the Outreach Agent had generated — with no test-specific prompting involved. That the failure mode was caught again, unprompted, in production is stronger evidence of the fix than the original test suite alone.

Calibration of the Governance Reviewer

Issue 2's fix was originally confirmed by 3/3 consistent runs on a single test draft plus one unprompted production catch. That is anecdotal evidence: it shows the reviewer can catch a grounding failure, not how often it does.

To measure it, the reviewer was calibrated against a 50-draft test set built from the actual approved brand sources and labeled by hand: 20 drafts where every claim is directly supported, 20 with a single planted contradiction (wrong model, wrong platform, wrong threshold, invented performance figures), and 10 with plausible-sounding but unverifiable additions — the harder category, since nothing in them is technically false, it simply is not backed by anything. Labeling policy: a draft passes only if every claim in it is directly supported; a single contradicted or ungrounded claim makes the whole draft a REVISE, the same rule the reviewer's own rubric enforces.

Result: 100% recall, 88.2% precision, 92% overall accuracy. The reviewer missed none of the 30 drafts that should have been flagged, including the exact "built for personal ritual, not standardized" phrasing from Issue 2 — confirming that fix holds under systematic testing rather than only in the original three runs. Four of twenty clean drafts were flagged unnecessarily.

Recall was treated as the metric to optimize: an unsupported claim reaching a customer is a brand and compliance problem, while an unnecessary flag costs a human reviewer a few seconds. Erring toward over-flagging is the correct direction for a compliance filter, and the 88.2% precision reflects a reviewer that reads paraphrase strictly rather than one that reasons loosely.

The calibration process itself required a correction worth noting. The first run returned 100% recall but only 68% precision, which initially looked like an over-strict reviewer. Inspecting the flagged drafts showed the cause was in the test, not the agent: several "clean" drafts contained personalization claims about lead behaviour, and the harness passed an empty lead context, so the reviewer had nothing to verify them against and correctly flagged them. The test set was rebuilt to isolate brand-source grounding alone, which is what the reviewer is actually responsible for. Verifying personalization claims against real lead data is a separate test, not yet run.

Issue 3 — JSON parsing edge case, and the structural fix

Asking the model to emit its verdict as free-form JSON text proved fragile in several ways: on longer claim decompositions the response could be truncated mid-string when it ran out of the token budget, and the model occasionally produced small escaping quirks — both breaking json.loads on an otherwise reasonable response. Rather than keep patching the

text parser defensively, the Governance Reviewer was migrated to tool use (function calling): the verdict schema is now declared as a tool the model must call, so the Anthropic API validates and returns a structured object directly, with no hand-written JSON parsing left in the code path. This eliminated the entire class of parsing failures rather than the single reported symptom — the same principle the Qualification Agent already used for its scoring tool.

Known limitations

The pipeline is verified against 3 fixed synthetic test leads, not a live, arbitrary lead stream — production deployment would need input validation for free-form leads. The human-approval step is a UI checkpoint only; no send integration exists, by design, for this portfolio. Governance evaluates against a fixed brand_sources.md; a real deployment would need a process for keeping that source of truth current as brand claims change. The governance calibration measures brand-source grounding only; whether the reviewer correctly verifies personalization claims against real lead data is untested. The 50-draft set is also fixed — a production system would need the test set to grow alongside brand_sources.md as claims change.

Skills Demonstrated and Next Steps

Skills demonstrated by this project

Category	Skills
Agentic AI	Tool use / function calling, multi-agent orchestration, structured JSON contracts between agents
AI evaluation & governance	Generation/evaluation separation, claim-level grounding verification, APPROVE/REVISE verdict design, LLM-as-judge calibration against a hand-labeled test set with precision/recall measurement
Retrieval-augmented generation	LangChain + FAISS, local sentence-transformer embeddings, chunking strategy, top-k retrieval
Machine learning	XGBoost as an agentic tool, SHAP explainability surfaced in natural-language reasoning
Prompt engineering	System prompts enforcing tool-first behavior, safe fallbacks on tool failure, forced decomposition to prevent evaluation shortcuts
Debugging & rigor	Diagnosed and fixed a model artifact mismatch, a governance grounding miss, and a JSON parsing edge case — each documented with root cause and fix
Deployment & production rigor	Streamlit multi-page integration, live verification on Streamlit Cloud including a real production REVISE catch; API rate limiting with exponential backoff, and a live observability dashboard tracking the weekly REVISE-vs-APPROVE governance split
Software engineering	Python 3.13, Anthropic Claude API, joblib model artifacts, Git versioning

Next steps (v1.1 and beyond)

The following extensions would deepen this project for v1.1:

• **Broaden observability.** Rate limiting with backoff and a weekly REVISE-vs-APPROVE observability dashboard are now in place; a next step is richer metrics — per-criterion flag rates and latency percentiles — alongside the current verdict trend.

• **Caching on the 3 test leads.** Since the demo leads are fixed, their agent outputs could be cached to reduce latency and API cost for repeat visitors, with a manual refresh option.

• **Extension to custom leads.** Add a form for arbitrary lead input with schema validation against models/feature_spec_v1.json, rather than only the 3 fixed demo leads.

Project trajectory in portfolio

P7 is the seventh and most recent project in the portfolio. Where P1 established the lead scoring model and its static segmentation table, P7 closes the loop: it takes that model off the page and puts it inside a live, auditable decisioning system — scoring, drafting, reviewing, and checkpointing an outreach decision end-to-end, with the same standard of honest disclosure applied to the agents' own failure modes as P1-P5 applied to their models and methods.

Contact

Sebastian Kradyel · Marketing Master's (9.54 GPA, Babeș-Bolyai University) · Reșița, Romania

Live demo: coffra-marketing-dashboard.streamlit.app · GitHub: github.com/sebikradyel1-svg